FIAP

Análise e Desenvolvimento de Sistemas

Documentação Final

Domain Driven Design using Java - 4º Sprint

Nome da solução

SoulUp

Nome da equipe

Grupo Dome

Integrantes

Arthur Cordão - RM570313

Guilherme Novaes - RM570444

Jeferson Faria - RM573038

Josué Menezes - RM569220

1. Objetivo e escopo do projeto

O **SoulUp** é uma plataforma de gamificação de ações sustentáveis. O usuário registra ações como reciclagem, recebe **Ecoa Points** e experiência (XP) para o seu avatar, conquista **Selos Verdes** e acumula **créditos de carbono**.

Esta entrega consiste na **API RESTful** da solução, desenvolvida em Java 21 com Spring Boot 4.1.1, que expõe todos os recursos do modelo de dados (Oracle) para consumo pelo front-end. O escopo cobre o CRUD completo de todas as entidades, as regras de negócio de pontuação, o tratamento padronizado de erros, a liberação de CORS e a publicação (deploy) da aplicação.

A arquitetura segue o padrão visto em aula, em camadas **Resource - BO - DAO - TO**: a camada Resource (pacote resource) recebe as requisições HTTP, a BO (pacote bo) aplica as regras de negócio, a DAO (pacote dao) acessa o banco por JDBC usando a classe ConnectionFactory, e os TOs (pacote to, Transfer Object) representam as tabelas do MER e validam os dados com Bean Validation.

2. Principais funcionalidades

- **Usuários:** cadastro, consulta, atualização e exclusão. E-mail único e senha armazenada com hash BCrypt (a senha nunca é devolvida pela API).
- **Avatar:** um avatar por usuário, com nível e XP.
- **Categorias e ações sustentáveis:** catálogo de ações com a pontuação Ecoa de cada uma e indicador de ativa/inativa.
- **Registro de ações:** o usuário registra uma ação realizada (status PENDENTE). Ao **aprovar**, a API credita os pontos no saldo do usuário e o XP no avatar, recalculando o nível (1 nível a cada 100 XP) em uma única transação; também é possível **rejeitar**.
- **Selos verdes e conquistas:** cadastro de selos e vínculo de selos a usuários, sem permitir duplicidade.
- **Créditos de carbono:** emissão e controle por usuário, com status ATIVO, UTILIZADO ou EXPIRADO.
- **Consultas por usuário:** avatar, ações, selos e créditos de um usuário em um único caminho de recurso.
- **Qualidade:** validações de entrada com Bean Validation (@Valid), códigos de status HTTP adequados, CorsFilter para o front-end e endpoint de verificação (/soulup).

3. Tabela de endpoints (API RESTful)

Todos os caminhos são relativos à URL base da aplicação e os corpos de requisição e resposta usam JSON. Requisições com corpo inválido (Bean Validation) retornam 400 automaticamente.

Verbo	URI	Descrição	Status esperados
GET	/soulup/usuario	Lista usuários	200, 500
GET	/soulup/usuario/{codigo}	Busca usuários pelo código	200, 404
POST	/soulup/usuario	Cadastra usuários (senha com hash BCrypt)	201, 400
PUT	/soulup/usuario/{codigo}	Atualiza usuários	201, 400, 404
DELETE	/soulup/usuario/{codigo}	Exclui usuários	204, 404, 409
GET	/soulup/usuario/{codigo}/avatar	Avatar do usuário	200, 404
GET	/soulup/usuario/{codigo}/acoes	Registros de ação do usuário	200, 404
GET	/soulup/usuario/{codigo}/selos	Selos conquistados pelo usuário	200, 404
GET	/soulup/usuario/{codigo}/creditos	Créditos de carbono do usuário	200, 404
GET	/soulup/avatar	Lista avatares	200, 500
GET	/soulup/avatar/{codigo}	Busca avatares pelo código	200, 404
POST	/soulup/avatar	Cadastra avatares	201, 400
PUT	/soulup/avatar/{codigo}	Atualiza avatares	201, 400, 404
DELETE	/soulup/avatar/{codigo}	Exclui avatares	204, 404, 409
GET	/soulup/categoria	Lista categorias de ação	200, 500
GET	/soulup/categoria/{codigo}	Busca categorias de ação pelo código	200, 404
POST	/soulup/categoria	Cadastra categorias de ação	201, 400
PUT	/soulup/categoria/{codigo}	Atualiza categorias de ação	201, 400, 404
DELETE	/soulup/categoria/{codigo}	Exclui categorias de ação	204, 404, 409
GET	/soulup/acao	Lista ações sustentáveis	200, 500
GET	/soulup/acao/{codigo}	Busca ações sustentáveis pelo código	200, 404
POST	/soulup/acao	Cadastra ações sustentáveis	201, 400
PUT	/soulup/acao/{codigo}	Atualiza ações sustentáveis	201, 400, 404
DELETE	/soulup/acao/{codigo}	Exclui ações sustentáveis	204, 404, 409
GET	/soulup/acao/categoria/{codigoCategoria}	Lista as ações de uma categoria	200, 500
GET	/soulup/registro-acao	Lista registros de ação	200, 500
GET	/soulup/registro-acao/{codigo}	Busca registros de ação pelo código	200, 404
POST	/soulup/registro-acao	Cadastra registros de ação (nasce PENDENTE, pontos vêm da ação)	201, 400
PUT	/soulup/registro-acao/{codigo}	Atualiza registros de ação	201, 400, 404
DELETE	/soulup/registro-acao/{codigo}	Exclui registros de ação	204, 404, 409
PATCH	/soulup/registro-acao/{codigo}/aprovar	Aprova o registro e credita pontos e XP	200, 400, 404
PATCH	/soulup/registro-acao/{codigo}/rejeitar	Rejeita o registro	200, 400, 404
GET	/soulup/selo	Lista selos verdes	200, 500
GET	/soulup/selo/{codigo}	Busca selos verdes pelo código	200, 404
POST	/soulup/selo	Cadastra selos verdes	201, 400

Verbo	URI	Descrição	Status esperados
PUT	/soulup/selo/{codigo}	Atualiza selos verdes	201, 400, 404
DELETE	/soulup/selo/{codigo}	Exclui selos verdes	204, 404, 409
GET	/soulup/conquista	Lista conquistas de selo	200, 500
GET	/soulup/conquista/{codigo}	Busca conquistas de selo pelo código	200, 404
POST	/soulup/conquista	Cadastra conquistas de selo	201, 400
PUT	/soulup/conquista/{codigo}	Atualiza conquistas de selo	201, 400, 404
DELETE	/soulup/conquista/{codigo}	Exclui conquistas de selo	204, 404, 409
GET	/soulup/credito-carbono	Lista créditos de carbono	200, 500
GET	/soulup/credito-carbono/{codigo}	Busca créditos de carbono pelo código	200, 404
POST	/soulup/credito-carbono	Cadastra créditos de carbono	201, 400
PUT	/soulup/credito-carbono/{codigo}	Atualiza créditos de carbono	201, 400, 404
DELETE	/soulup/credito-carbono/{codigo}	Exclui créditos de carbono	204, 404, 409
GET	/soulup	Verifica se a API está no ar	200

Significado dos códigos: 200 OK (requisição atendida); 201 Created (recurso criado ou atualizado com sucesso, conforme o padrão adotado em aula); 204 No Content (excluído com sucesso); 400 Bad Request (dados inválidos ou regra de negócio violada); 404 Not Found (recurso não encontrado); 409 Conflict (exclusão impedida por registros relacionados); 500 Internal Server Error (falha no servidor ou no banco).

3.1 Exemplos de requisição e resposta

POST /soulup/usuario - 201 Created

```
{
  "nome": "Maria Silva",
  "email": "maria@soulup.io",
  "senha": "senha123",
  "dataNascimento": "2000-04-15"
}

-> {
  "codigo": 2,
  "nome": "Maria Silva",
  "email": "maria@soulup.io",
  "senha": null,
  "dataNascimento": "2000-04-15",
  "dataCadastro": "2026-10-23T21:10:00",
  "saldoEcoaPoints": 0
}
```

POST /soulup/registro-acao - 201 Created

```
{
  "codigoUsuario": 1,
  "codigoAcao": 1,
  "evidenciaUrl": "https://exemplo.com/foto.jpg"
}

-> {
  "codigo": 1,
  "codigoUsuario": 1,
  "codigoAcao": 1,
  "dataRealizacao": "2026-10-23T21:12:00",
  "pontosGanhos": 50,
  "evidenciaUrl": "https://exemplo.com/foto.jpg",
  "status": "PENDENTE"
}
```

PATCH /soulup/registro-acao/1/aprovar - 200 OK

```
-> {
  "codigo": 1,
  "codigoUsuario": 1,
  "codigoAcao": 1,
  "pontosGanhos": 50,
  "status": "APROVADO", ...
}
```

GET /soulup/usuario/99 - 404 Not Found

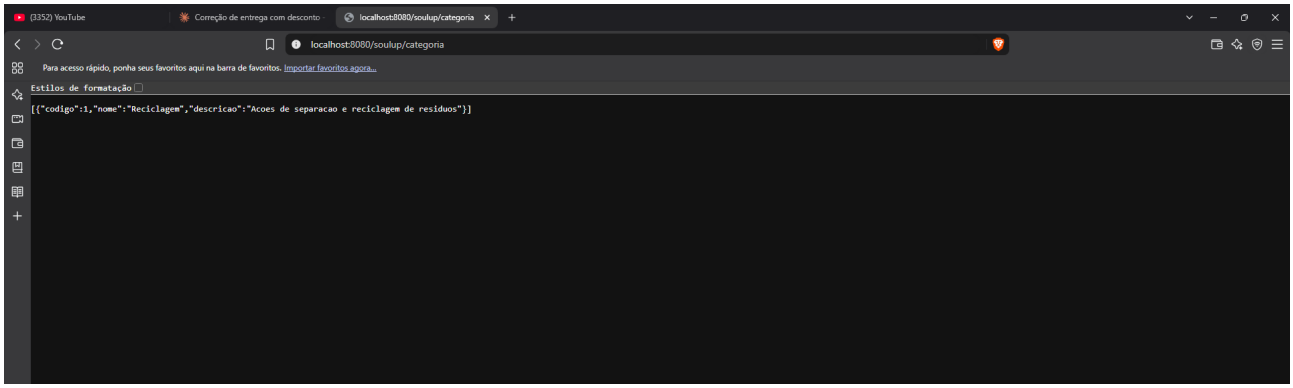
```
Usuário não encontrado!
```

3.2 Principais validações e regras de negócio

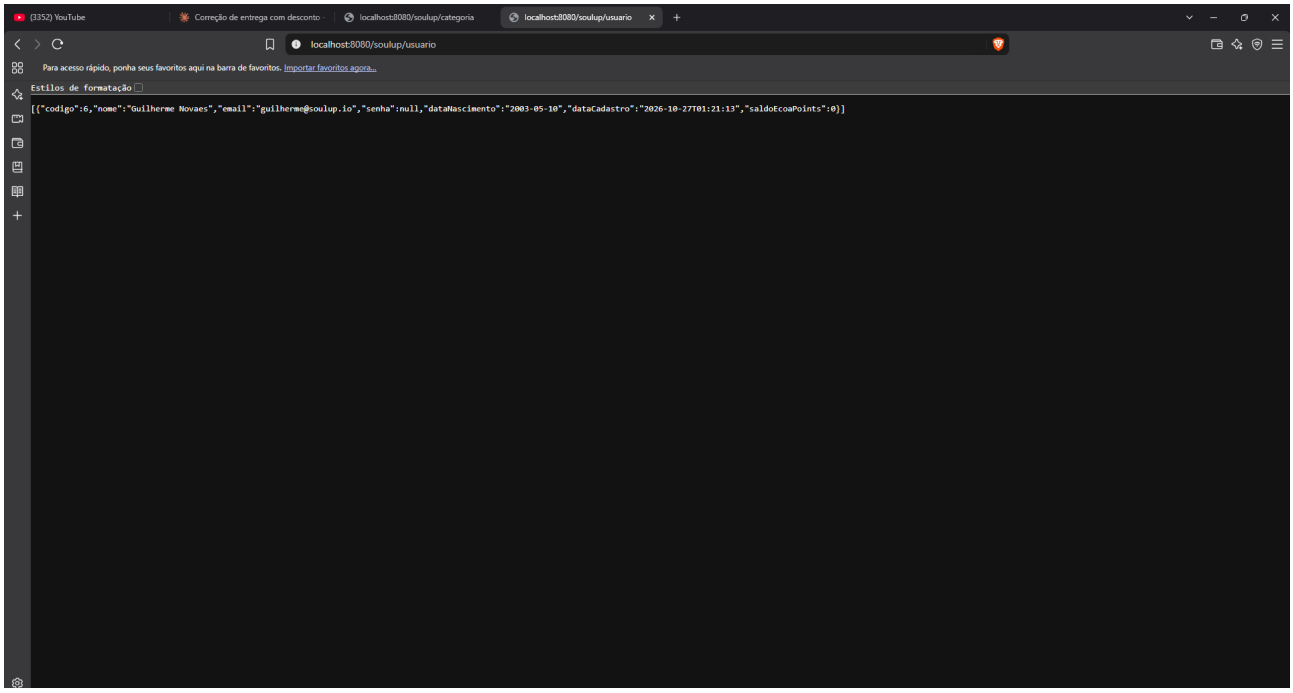
Recurso	Regras
Usuário	Nome obrigatório (até 120); e-mail válido e único; senha de 6 a 72 caracteres; data de nascimento não pode estar no futuro.
Avatar	Usuário deve existir e ter no máximo um avatar; nível mínimo 1 (padrão 1); XP não negativo (padrão 0).
Ação sustentável	Categoria deve existir; pontos Ecoa maiores que zero; ativa por padrão.
Registro de ação	Usuário e ação devem existir e a ação deve estar ativa; pontos copiados da ação; só registros PENDENTES podem ser alterados, aprovados ou rejeitados; aprovados não podem ser excluídos.
Conquista de selo	Usuário e selo devem existir; o mesmo selo não pode ser conquistado duas vezes pelo mesmo usuário.
Crédito de carbono	Quantidade maior que zero (2 casas decimais); origem obrigatória; status ATIVO, UTILIZADO ou EXPIRADO.
Bean Validation	Os TOs usam @NotBlank, @NotNull, @Size, @Email, @Past, @Positive, @Min e @Pattern; o @Valid nos métodos save e update devolve 400 para dados inválidos.

4. Protótipo - prints do sistema em funcionamento

Capturas de tela reais da API em funcionamento: testes locais dos endpoints e deploy publicado no Render, cada uma com a explicação do que mostra.



Teste local: GET soulup categoria retorna a lista vinda do Oracle



Teste local: GET soulup usuario lista os usuarios sem expor a senha



POST soulup usuario cria o usuario e retorna os dados cadastrados



POST soulup registro-acao cria o registro com status PENDENTE e 50 pontos



GET soulup usuario 7 mostra o saldo zerado antes da aprovacao



PATCH soulup registro-acao 1 aprovar muda o status para APROVADO



GET soulup usuario 7 mostra o saldo de 50 pontos apos a aprovacao



POST soulup avatar cria o avatar com nivel 1 e xp 0



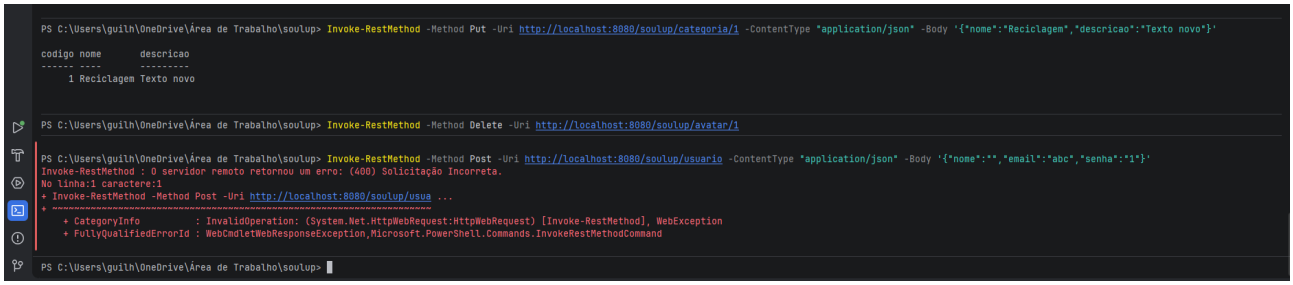
POST soulup registro-acao cria o segundo registro PENDENTE



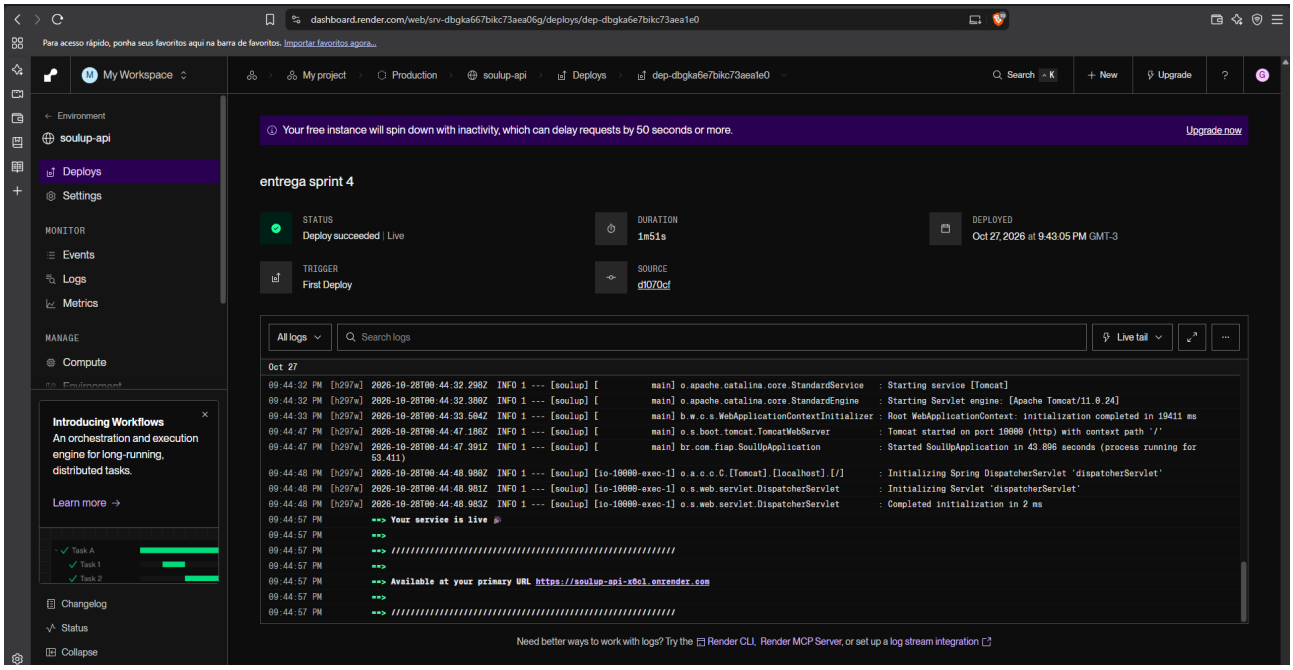
Aprovar registro ja processado e recusado e o avatar acumula 50 de xp



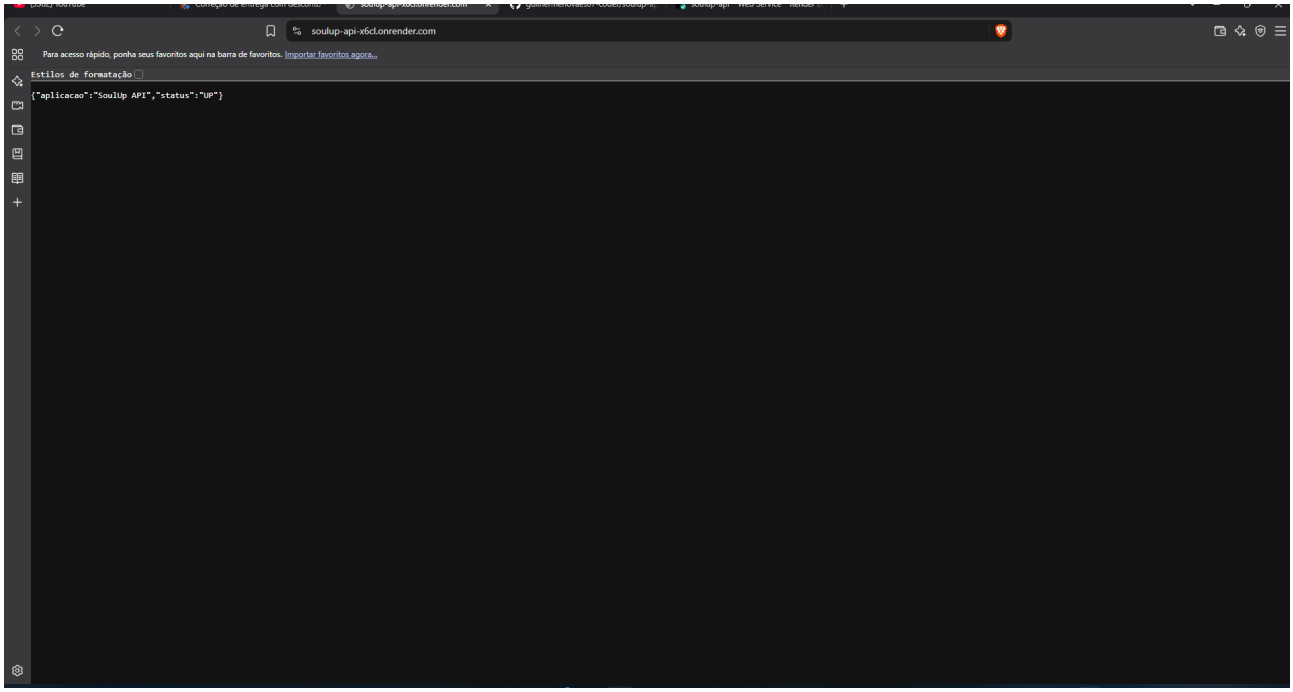
GET soulup usuario 7 mostra o saldo de 100 pontos



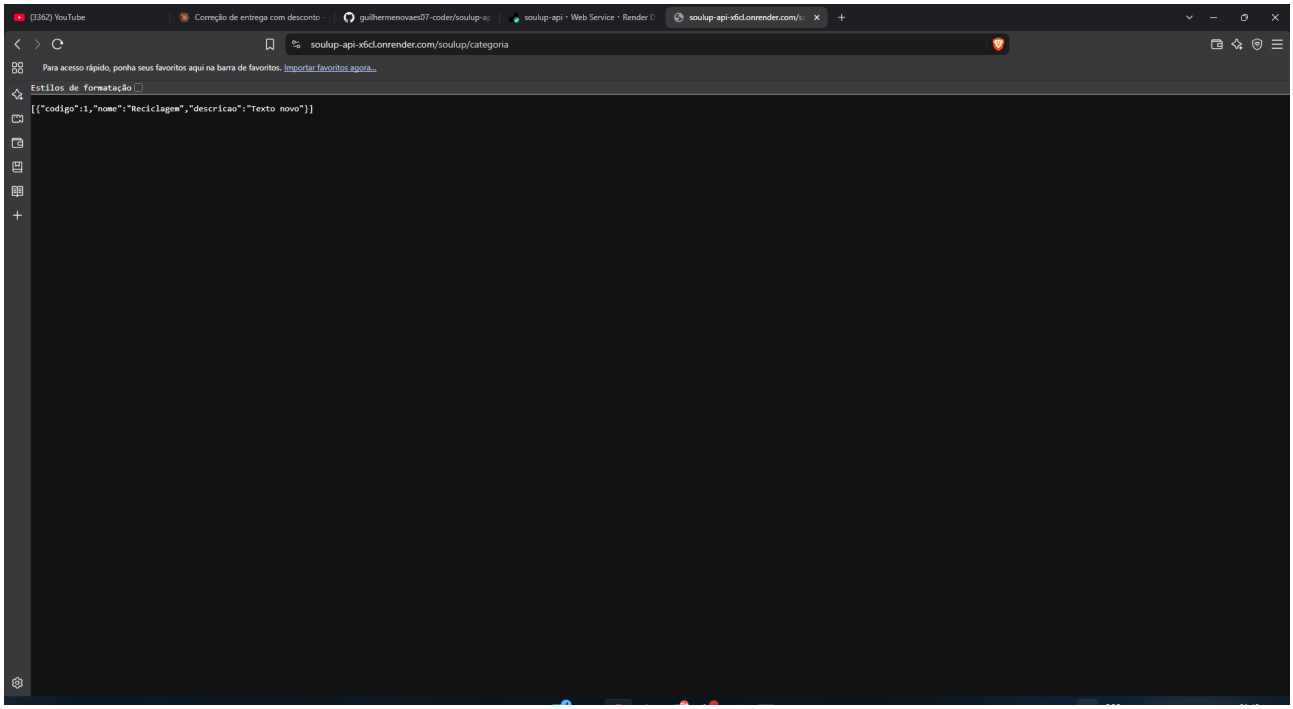
PUT de categoria, DELETE de avatar e erro 400 na validacao de dados invalidos



Render: deploy concluido com sucesso e servico no ar



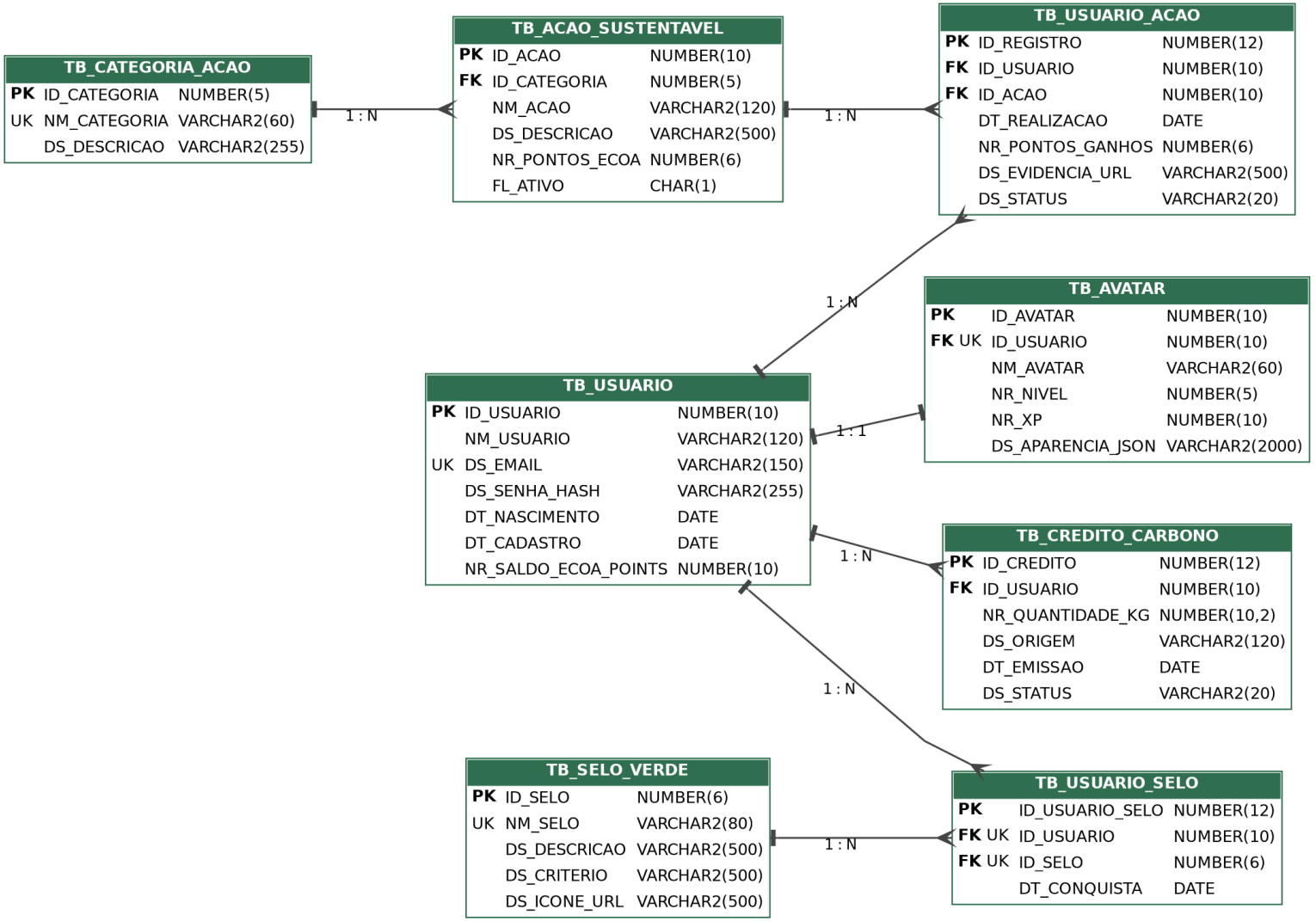
Deploy: GET soulup responde status UP no endereco do Render



Deploy: GET soulup categoria retorna dados do Oracle pelo Render

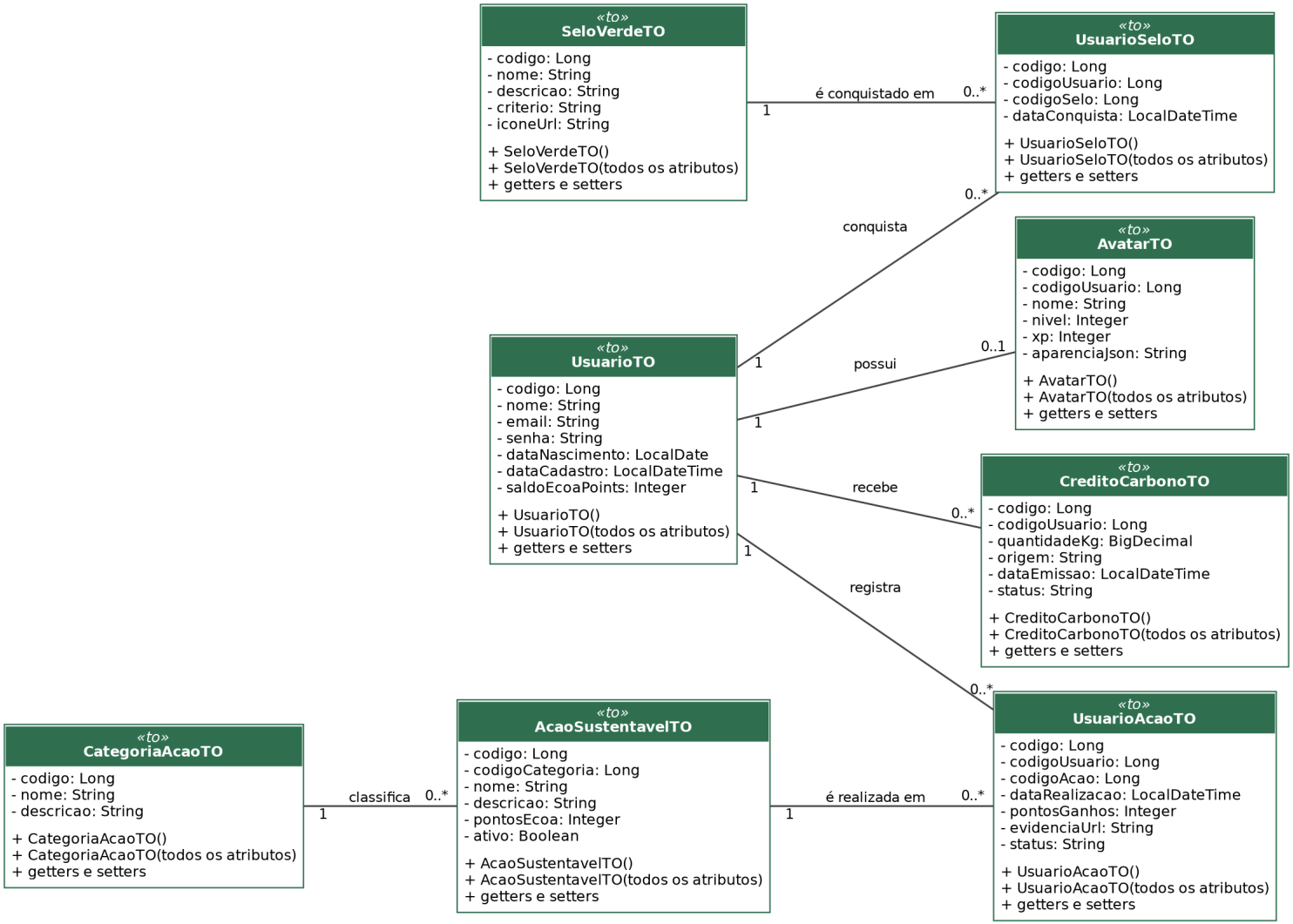
5. Modelo de Entidade-Relacionamento (MER)

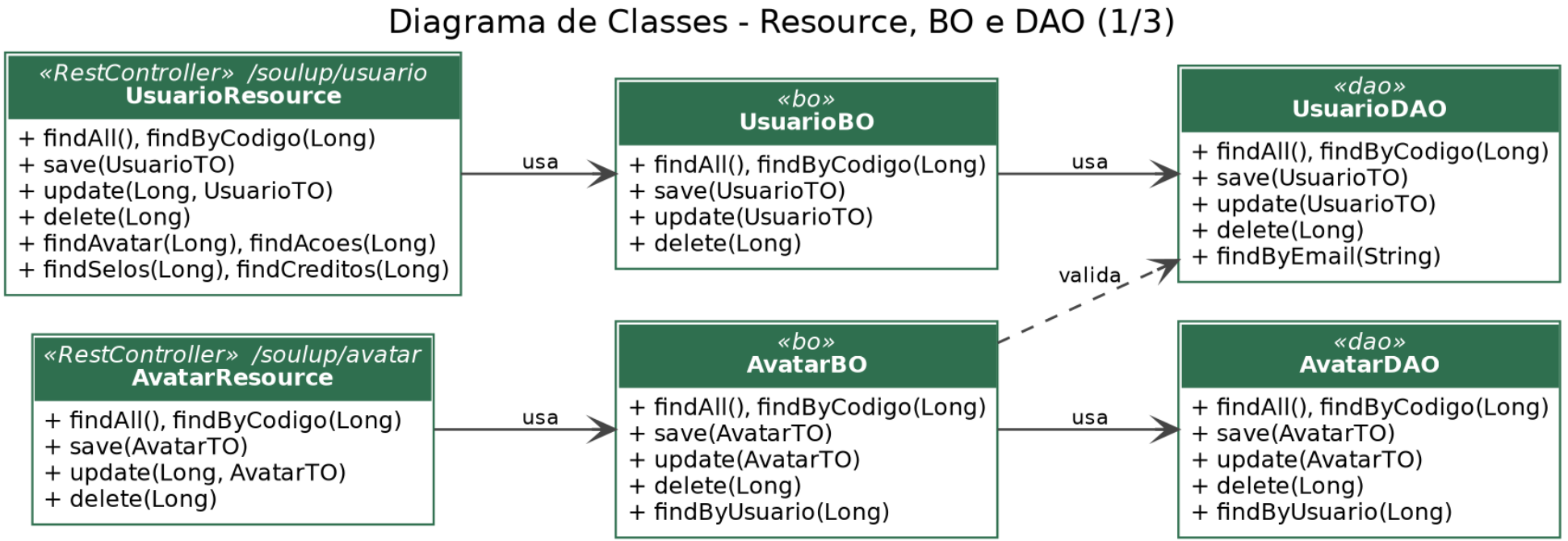
Diagrama físico do banco Oracle: tabelas, atributos, chaves primárias (PK), estrangeiras (FK), únicas (UK) e cardinalidades dos relacionamentos.

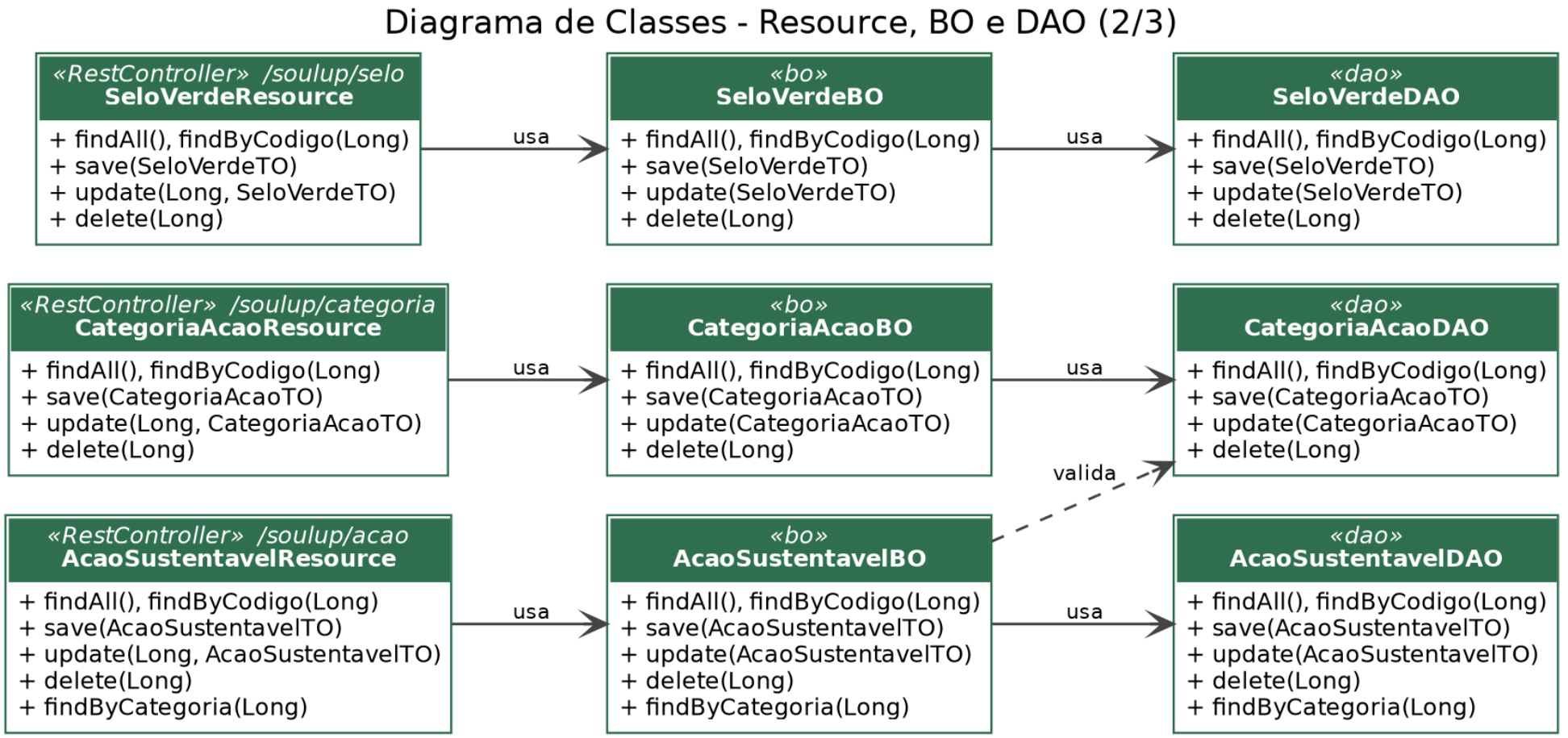


6. Diagrama de Classes atualizado

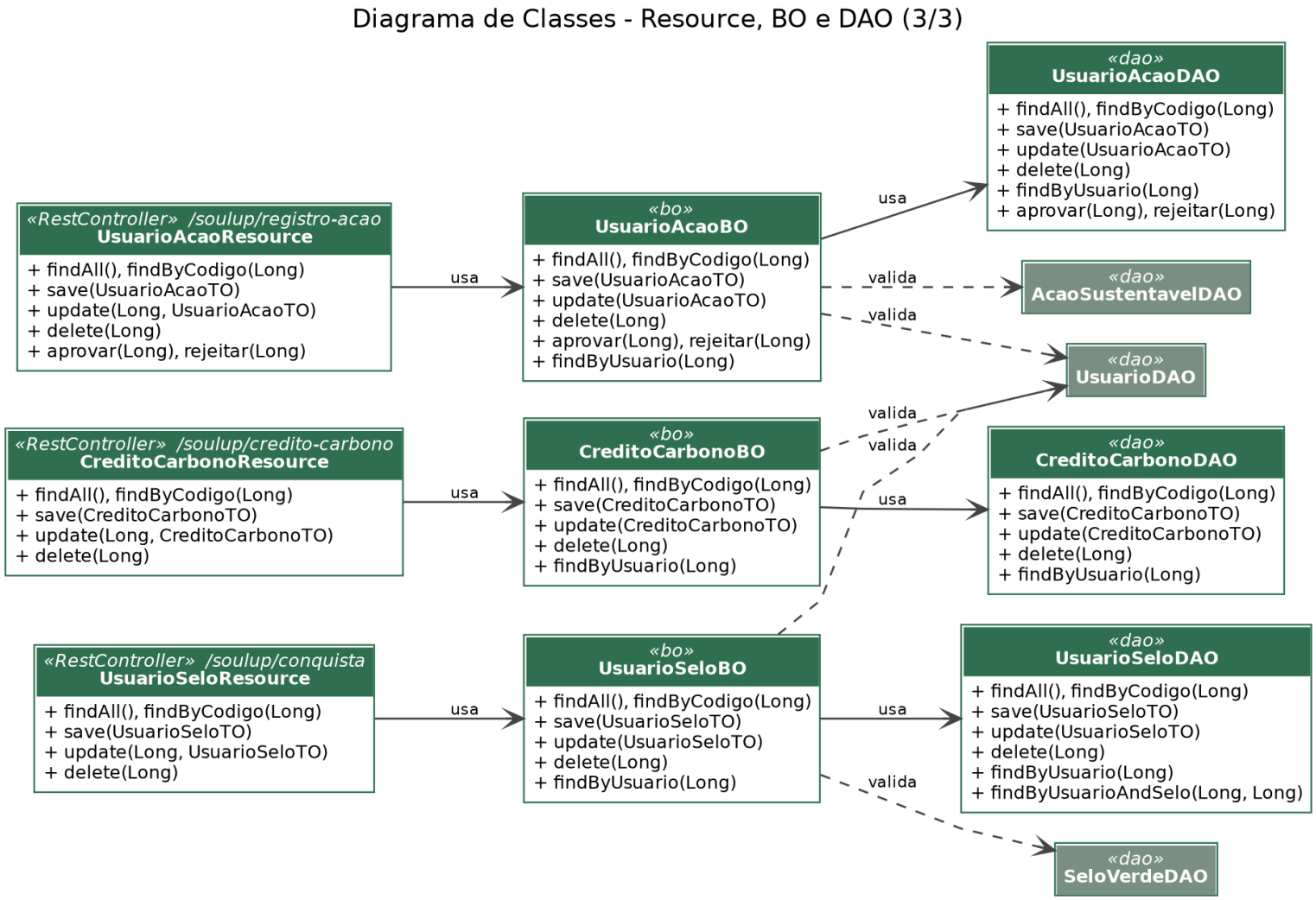
6.1 Camada TO (Transfer Object) - atributos, métodos e associações entre as classes.



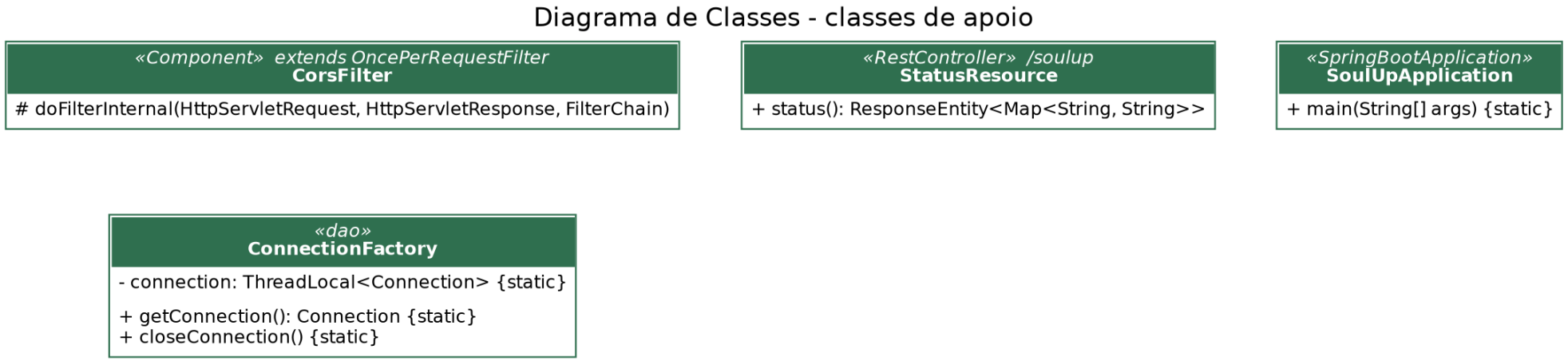

Diagrama de Classes - Resource, BO e DAO (1/3)
«RestController» /soulup/usuario
UsuarioResource
+ findAll(), findByCodigo(Long)
+ save(UsuarioTO)
+ update(Long, UsuarioTO)
+ delete(Long)
+ findAvatar(Long), findAcoes(Long)
+ findSelos(Long), findCreditos(Long)
usa
«bo»
UsuarioBO
+ findAll(), findByCodigo(Long)
+ save(UsuarioTO)
+ update(UsuarioTO)
+ delete(Long)
usa
«dao»
UsuarioDAO
+ findAll(), findByCodigo(Long)
+ save(UsuarioTO)
+ update(UsuarioTO)
+ delete(Long)
+ findByEmail(String)
valida
«RestController» /soulup/avatar
AvatarResource
+ findAll(), findByCodigo(Long)
+ save(AvatarTO)
+ update(Long, AvatarTO)
+ delete(Long)
usa
«bo»
AvatarBO
+ findAll(), findByCodigo(Long)
+ save(AvatarTO)
+ update(AvatarTO)
+ delete(Long)
+ findByUsuario(Long)
usa
«dao»
AvatarDAO
+ findAll(), findByCodigo(Long)
+ save(AvatarTO)
+ update(AvatarTO)
+ delete(Long)
+ findByUsuario(Long)


Diagrama de Classes - Resource, BO e DAO (2/3)
«RestController» /soulup/selo
SeloVerdeResource
+ findAll(), findByCodigo(Long)
+ save(SeloVerdeTO)
+ update(Long, SeloVerdeTO)
+ delete(Long)
usa
«bo»
SeloVerdeBO
+ findAll(), findByCodigo(Long)
+ save(SeloVerdeTO)
+ update(SeloVerdeTO)
+ delete(Long)
usa
«dao»
SeloVerdeDAO
+ findAll(), findByCodigo(Long)
+ save(SeloVerdeTO)
+ update(SeloVerdeTO)
+ delete(Long)
«RestController» /soulup/categoria
CategoriaAcaoResource
+ findAll(), findByCodigo(Long)
+ save(CategoriaAcaoTO)
+ update(Long, CategoriaAcaoTO)
+ delete(Long)
usa
«bo»
CategoriaAcaoBO
+ findAll(), findByCodigo(Long)
+ save(CategoriaAcaoTO)
+ update(CategoriaAcaoTO)
+ delete(Long)
usa
«dao»
CategoriaAcaoDAO
+ findAll(), findByCodigo(Long)
+ save(CategoriaAcaoTO)
+ update(CategoriaAcaoTO)
+ delete(Long)
valida
«RestController» /soulup/acao
AcaoSustentavelResource
+ findAll(), findByCodigo(Long)
+ save(AcaoSustentavelTO)
+ update(Long, AcaoSustentavelTO)
+ delete(Long)
+ findByCategoria(Long)
usa
«bo»
AcaoSustentavelBO
+ findAll(), findByCodigo(Long)
+ save(AcaoSustentavelTO)
+ update(AcaoSustentavelTO)
+ delete(Long)
+ findByCategoria(Long)
usa
«dao»
AcaoSustentavelDAO
+ findAll(), findByCodigo(Long)
+ save(AcaoSustentavelTO)
+ update(AcaoSustentavelTO)
+ delete(Long)
+ findByCategoria(Long)

6.4 Camadas Resource, BO e DAO - Registro de Ação, Conquista de Selo e Crédito de Carbono.



6.5 Classes de apoio - ConnectionFactory (usada por todas as DAOs), CorsFilter e demais.



7. Procedimentos para rodar a aplicação

7.1 Ferramentas e versões necessárias

Item	Ferramenta / versão
Linguagem	Java JDK 21
Build	Maven Wrapper (./mvnw, já incluído no projeto)
Framework	Spring Boot 4.1.1 (Spring Web, Validation e DevTools)
Banco de dados	Oracle (script em db/soulup_ddl.sql); driver ojdbc11
IDE	IntelliJ IDEA (também funciona em Eclipse ou VS Code)
Apoio	Oracle SQL Developer (executar o script) e Insomnia (testar os endpoints)

7.2 Passo a passo da execução

1. Clone o repositório do GitHub e abra a pasta do projeto.
2. No Oracle SQL Developer, conectado ao servidor, execute o script **db/soulup_ddl.sql** (cria as tabelas, sequences e dados de teste).
3. Configure as variáveis de ambiente **DB_URL** (jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL), **DB_USER** e **DB_PASSWORD**. No IntelliJ: Run > Edit Configurations > Environment variables.
4. Execute a classe **SoulUpApplication** (botão Run). O Tomcat embutido inicia na porta 8080.
5. Teste em `http://localhost:8080/soulup`; a resposta deve conter o status UP. Em seguida, use o Insomnia nos endpoints da tabela da seção 3.
6. Para gerar o executável: `./mvnw clean -DskipTests package` e execute `java -jar target/soulup-0.0.1-SNAPSHOT.jar`.

7.3 Como importar o projeto

- **IntelliJ IDEA:** File > Open, selecione o arquivo **pom.xml** e escolha Open as Project; aguarde o Maven carregar as dependências.
- **Eclipse:** File > Import > Maven > Existing Maven Projects e selecione a pasta do projeto.
- **VS Code:** File > Open Folder na pasta do projeto, com o Extension Pack for Java instalado.

7.4 Deploy

A aplicação é publicada no **Render** como Web Service, usando o **Dockerfile** da raiz do projeto. As credenciais do banco não ficam no código: são lidas das variáveis de ambiente DB_URL, DB_USER e DB_PASSWORD, configuradas no painel do Render. Passos: subir o projeto no GitHub, criar o Web Service apontando para o repositório (plano gratuito), preencher as variáveis de ambiente e clicar em Deploy Web Service. A URL pública do deploy é informada na entrega.